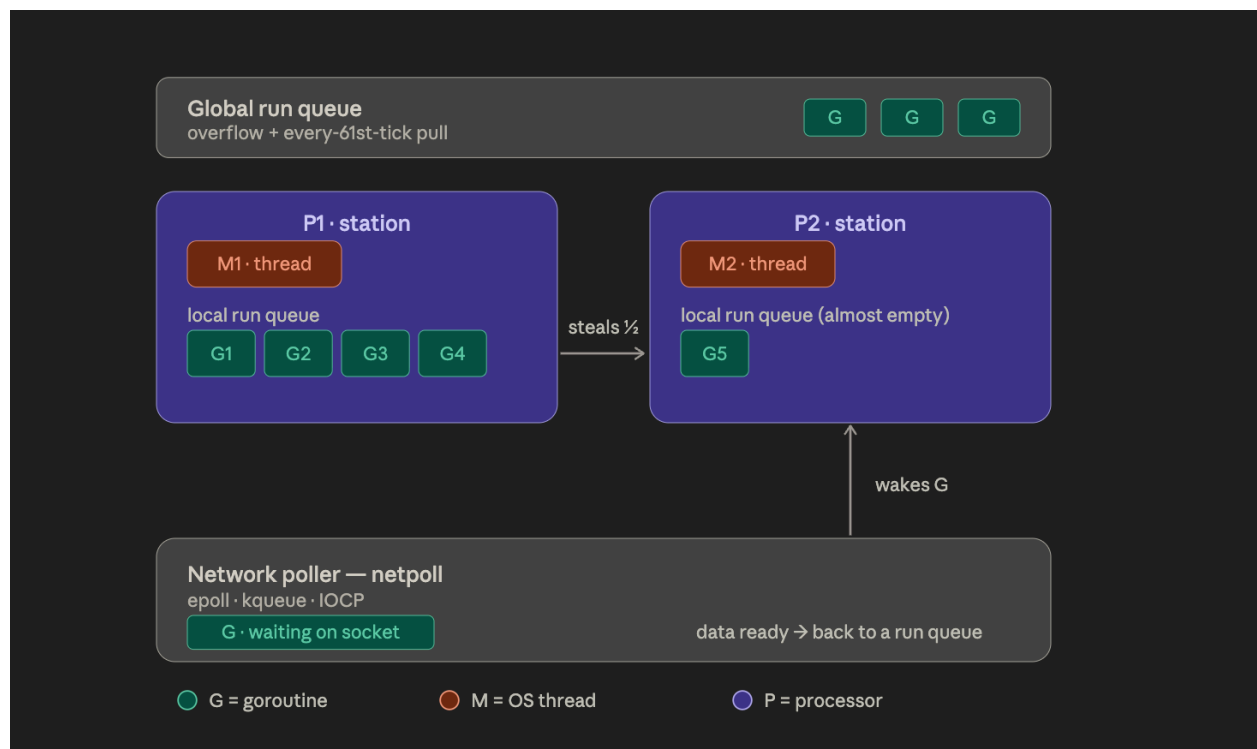


This is a great thing to go deep on, and you've already got a solid scaffold from that ChatGPT thread. Let me rebuild it from the ground up around one concrete **for** loop and a single analogy I'll carry the whole way through — a **restaurant kitchen** — because every piece of the Go scheduler maps cleanly onto it.

Let me get one diagram ready to anchor the spatial model, then we'll trace the loop instruction by instruction. Here's the spatial model. Keep this picture in your head — everything below refers back to it. The kitchen: **G** = a dish/order ticket (the work), **M** = a chef (an OS thread, the only thing that can actually cook), **P** = a cooking station with its own ticket rail (there are only **GOMAXPROCS** of them). Now let me build the whole thing from the one idea everything hangs on.



## The one reframe that makes it all click

When you write `go worker()`, you are **not** asking the operating system for a thread. You're dropping an order ticket onto a queue that belongs to Go's *own* scheduler — a tiny operating system that lives inside your program, in user space, on top of the real OS. That's the whole trick. Go runs its own little restaurant on top of the kernel's restaurant.

Why bother? Because an OS thread is an expensive chef to hire. Each one reserves around a megabyte of stack and carries kernel bookkeeping, and every time the CPU switches from one

thread to another it has to dive into the kernel, save registers, and swap context — costly. A goroutine starts at about **2 KB** of stack, and switching between two goroutines is just Go stashing a few registers and a stack pointer in user space, no kernel trip at all. That's the difference between "I can have eight chefs" and "I can have a million dishes in flight." The job of the scheduler is to multiplex those millions of dishes (G) onto a handful of chefs (M). People call this **M:N scheduling** — M goroutines run on N threads.

To do that bookkeeping, Go uses three structures, the famous **G-M-P**:

- **G (goroutine)** — the dish/ticket. It's *just a struct*: it holds its own stack, a program counter (which line it's paused at), its status, and some metadata. A G does nothing on its own — it's a description of work plus the saved state of that work.
- **M (machine)** — an actual OS thread. The chef. The *only* thing that can physically execute code, because the CPU only ever runs threads. An M with no work parks itself (sleeps).
- **P (processor)** — *not* a CPU and *not* a thread. It's a "license to run Go code" bundled with a local run queue and some caches. The cooking station. Go creates exactly **GOMAXPROCS** of them, which defaults to your number of cores.

The rule that ties it together: **to run a goroutine, a chef must be standing at a station. M + P + G = execution.** No free station, no cooking — even with idle chefs and a pile of tickets. *That* is why **GOMAXPROCS** caps how many goroutines run at literally the same instant. You can have a million Gs and ten thousand Ms, but only **GOMAXPROCS** dishes are being actively cooked at any single moment.

## The for loop, traced instruction by instruction

Here's the example you asked for. Two cores, so two Ps:

```
func main() {  
    for i := 0; i < 10; i++ {  
        go worker(i)  
    }  
    time.Sleep(time.Second)  
}
```

First thing to internalize: **main** is *itself* a goroutine. Call it G-main. It's currently running on, say, M1 standing at P1.

Now the loop runs. When **go worker(0)** executes, here is what mechanically happens — and notice what does *not*:

1. The runtime grabs a G struct (it keeps a free-list of dead Gs and reuses them — creating a goroutine usually allocates nothing).
2. It gives that G a fresh 2 KB stack, sets its program counter to the first instruction of `worker`, and stashes the argument `0`.
3. It marks the G **runnable** and pushes it onto **P1's local run queue**.
4. It returns immediately and the loop continues.

The goroutine does **not** start running. `go` is not "run this now in parallel" — it's "put this ticket on the rail and keep moving." The loop is blisteringly fast because all it does is mint structs and stack them: `worker(0)`, `worker(1)`, ... land one after another on P1's rail. G-main is still the only thing actually executing, and it's heading for `Sleep`.

A detail worth knowing: the local run queue is a **fixed-size ring buffer of 256 slots**. If it overflows, the runtime flushes *half* of it up to the **global run queue**. So in your real code — `for i := 0; i < 1_000_000; i++` — the first ~256 land on P1's rail and the rest spill to the global board. Millions of goroutines don't mean millions of busy structures; they mean a couple of queues holding runnable Gs and the vast majority sitting parked.

Then G-main hits `time.Sleep`. Sleep **parks** the goroutine: G-main registers a timer, flips to *waiting*, and yields the station. M1 now has a station (P1) and nothing to run, so it calls into the scheduler, which pops `worker(0)` off P1's local queue and starts executing it. Meanwhile P2 has an empty rail. Its chef (M2, woken up or freshly minted) looks around, sees P1 holding a pile, and **steals roughly half** of P1's queue over to P2. Now both stations cook in parallel.

That single loop demonstrates the entire core: cheap creation (structs onto a queue), deferred scheduling (nothing runs until a chef is free), real parallelism (two stations at once), and automatic load balancing (stealing). Everything below is just the runtime handling the awkward cases.

## Work stealing — keeping every chef busy

You saw it above. The subtlety is the *order* a hungry scheduler looks for work, because that order prevents starvation:

1. Its own local queue first (cheapest — no locks contended).
2. Every 61st scheduling tick, it deliberately checks the **global queue** first instead. This odd-looking rule exists so the global board doesn't starve: if local queues always have work, nothing would ever pull from global without this forced peek.
3. The **network poller** (more in a moment).
4. Only then does it **steal** half of a random other P's local queue.

The chef checks their own rail, occasionally checks the central board, checks whether any oven has dinged, and only then walks over and grabs tickets off a busy colleague's rail.

## Context switching — why goroutine switches are cheap

When the scheduler swaps `worker1` out for `worker2`, it saves into the G struct just the program counter, the stack pointer, and a few registers — all in user space. No kernel involvement, no page-table juggling. Compared to an OS thread switch (a syscall, a kernel-mode transition), this is an order of magnitude cheaper. That cheapness is *why* Go can afford to switch constantly and run millions of goroutines without the machine grinding to a halt.

## Preemption — the infinite-loop problem, and how Go fixed it

Consider the nightmare case:

```
go func() { for {} }()
```

This loop never calls a function, never touches a channel, never allocates — it never voluntarily re-enters the runtime. In the old days (before Go 1.14) the scheduler could only preempt a goroutine at function-call boundaries, where the compiler had inserted a tiny check in the function prologue. A tight loop with no calls had **no such check**, so it would hog its station forever. Worse, garbage collection needs every goroutine to pause at a safe point, so one runaway loop could stall the whole program's GC.

Since Go 1.14, Go has **asynchronous preemption**. A supervisor (next section) notices a goroutine has been running too long — about 10 ms — and sends the chef's thread an OS signal (`SIGURG` on Unix). The signal handler safely parks the running G at a valid point and reschedules. The manager walks over, taps the chef who's been monopolizing the stove, and says "enough — let someone else cook." This is exactly why `for {}` no longer freezes a modern Go program.

## sysmon — the manager nobody mentions

That supervisor is `sysmon`, a special M that runs **without a P** — it doesn't need a station because it doesn't cook, it just watches. It loops forever, sleeping somewhere between 20 μs and 10 ms between rounds, and it handles the things that fall through the cracks: triggering the preemption above, retaking Ps from goroutines stuck in long syscalls, poking the network poller if it's been neglected, forcing periodic GC. It's the floor manager pacing the kitchen making sure nothing seizes up.

## Network poller — how one server holds a million connections

This is the piece that makes people from a Node.js background finally get Go. Say `worker` does:

```
n, err := conn.Read(buf)
```

and no data has arrived yet. The naive design would be "the chef stands at the oven staring at it" — the thread blocks, the station is occupied, and a CPU is wasted doing nothing. Go refuses to do that. Instead:

- It registers `conn`'s file descriptor with the OS's readiness mechanism — `epoll` on Linux, `kqueue` on macOS/BSD, `IOCP` on Windows.
- It **parks** the G (status: waiting) and — critically — **frees the M and P** to go run other goroutines.

The chef slides the dish into the oven, sets a timer, and immediately goes back to cooking other tickets at the station. When the OS reports the socket is readable, the poller flips that G back to **runnable** and drops it on a run queue; a chef picks it up and it resumes **on the exact line after `conn.Read`**, with every local variable intact.

So a waiting connection costs you a parked goroutine — a few KB — not an entire OS thread. *That* is the answer to "how does one Go server handle a million idle connections with a handful of threads": idle connections are cheap parked Gs, and the threads are only ever busy with sockets that actually have data right now. This is the engine under `net/http`, gRPC servers, database drivers — the things you build at Probus.

## Blocking syscalls — a different beast from network I/O

Watch out for this distinction, because it trips people up. The poller only helps with things the OS can watch for *readiness* — mostly network sockets. A genuinely blocking syscall — `os.Open`, a synchronous file read, a CGo call — actually pins the OS thread; there's nothing to poll. So Go does a **handoff**:

When an M is about to enter a blocking syscall, the runtime detaches its P first. The M dives into the syscall carrying its G with it (both are stuck for the duration). The now-orphaned P is immediately handed to another M — woken from the idle pool or freshly created — so that station keeps cooking. When the syscall finally returns, the original M tries to reacquire a P; if none is free, it parks its G on the global queue and goes to sleep. The chef who has to make a long trip to the walk-in freezer hands their station off before leaving, so the kitchen never stalls.

The flip side: lots of *simultaneous* blocking syscalls force Go to spin up lots of OS threads (one stuck per call). That's usually fine, but it's why a service doing heavy synchronous file or CGo work can quietly accumulate hundreds of threads while a network-bound service stays lean.

## Channels — the pass window between stations

A channel isn't magic; it's a struct holding a ring-buffer, a lock, a queue of waiting senders, and a queue of waiting receivers. For an unbuffered `ch <- x`:

- If a receiver is already parked waiting, the runtime copies the value straight to it and wakes that receiver's G — the value never touches a buffer. A direct hand-off through the pass window.
- If no receiver is waiting, the *sender's* G parks itself on the send-wait queue and gets woken later when a receiver shows up.

A buffered channel just copies into the buffer and keeps going while there's room, only parking the sender when the buffer is full. The key insight: "blocking on a channel" never means a busy-wait or a wasted thread — it's the **same parking mechanism** as sleep and netpoll. Your G goes to *waiting*, your M is freed for other work. This is why channel-heavy Go code scales: blocking is nearly free.

## Stack growth — why 2 KB is a safe starting point

Each goroutine starts with a 2 KB stack. When a function call would overflow it, a check in the function prologue catches it; the runtime allocates a **new, larger stack** (it doubles: 2→4→8→16 KB...), copies the old contents over, fixes up any pointers into the stack, and continues as if nothing happened. Stacks can shrink again during GC. This "contiguous, growable stack" is precisely what lets a goroutine be cheap by default — you only pay for a big stack if the goroutine actually goes deep — and it's the final ingredient that makes a million goroutines feasible.

## The lifecycle, all in one place

Tie the states together and the whole thing becomes legible:

```
go worker()
↓
_Grunnable (struct minted, sitting on a queue)
↓      ← a chef (M+P) picks it up
_Grunning (executing on a thread)
↓      ← channel / sleep / netpoll / mutex
_Gwaiting (parked; M is freed for other work)
↓      ← or a blocking syscall → _Gsyscall (M pinned, P handed off)
↓      ← unblocked: poller/timer/sender wakes it
_Grunnable (back on a queue, resumes from the exact paused line)
↓
_Gdead (worker returned; struct recycled, not freed)
```

When you debug a hung production service, the question is almost always "**why is this G stuck in `_Gwaiting`, and who was supposed to wake it?**" A receiver that never came, a lock never released, a context never cancelled.

## The scheduler loop and the Node comparison

Every chef, after finishing or parking a dish, runs the same loop forever: find a runnable G (local → occasionally global → poller → steal), run it, and when it blocks, save its state and find the next one. That loop is the heartbeat of the runtime.

The Node.js contrast sharpens the whole model. Node has one JS thread and one event loop; an async operation hands over a *callback*, and later the event loop runs that callback — control is inverted, and your logic gets shredded across `.thens` or `async/await` desugaring. Go instead saves the goroutine's entire stack and program counter, so when the I/O completes it **resumes from the exact next instruction** with all locals alive. You write plain top-to-bottom code that *looks* synchronous, and the runtime invisibly handles every pause and resume. Node says "when ready, run this callback"; Go says "when ready, wake this goroutine and continue mid-sentence." That's why goroutine code reads so much more naturally than callback-driven code while scaling just as well.

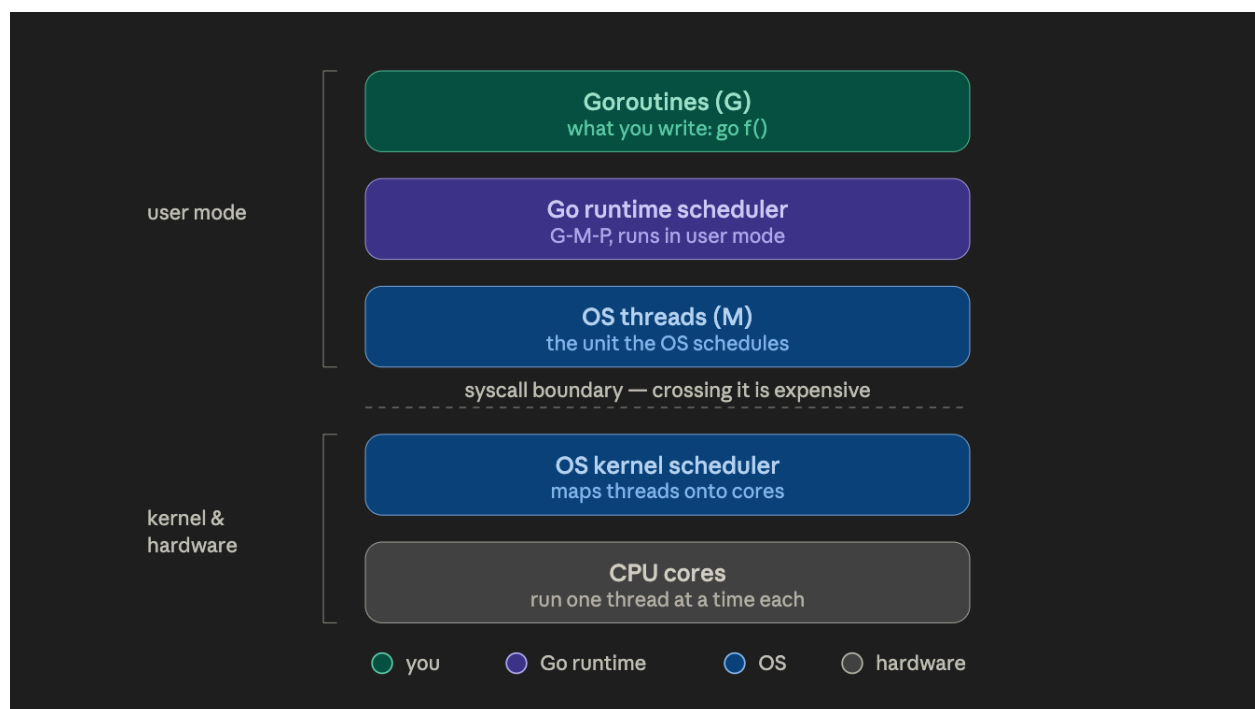
---

That's the runtime from the metal up: process and thread underneath, then G-M-P, queues, stealing, context switches, preemption via sysmon, the network poller, syscall handoff, channels, stack growth, the lifecycle, and the scheduler loop — all the same handful of moves applied to different situations.

The layers your notes list *above* this — memory model and happens-before, escape analysis, heap vs stack, `select` internals, mutex fast/slow paths, atomics and CAS, `context` propagation, GC tri-color marking, and the profiling tools (`pprof`, `go tool trace`) — all sit on top of this foundation and make far more sense once the scheduler is solid in your head, so they're the natural next stops.

One honest nudge, since you go deep on this stuff: the fastest way to *cement* all of the above is to stop reading about it and watch it happen. Run any of your goroutine programs with `GODEBUG=schedtrace=1000,scheddetail=1` and you'll see the Ps, run-queue lengths, and Ms printed live every second — work stealing and queue spillover become visible rather than theoretical. `go tool trace` will literally draw the goroutines moving across Ps. An afternoon staring at that on your bulk-email worker pool will teach you more than another explainer. Want me to walk through reading a `schedtrace` dump from one of your own programs?

Yes — you've got the right instinct, and those three (**core, process, thread**) are exactly the bottom of the ladder. But here's the thing: the *nouns* (core/process/thread) are only half of it. The other half is the *mechanisms* that connect them — and those are the real prerequisites, because the entire Go scheduler exists to work around their costs. Let me give you the full ladder first, then walk each rung.



Yes — you've got the right instinct, and those three (**core, process, thread**) are exactly the bottom of the ladder. But here's the thing: the *nouns* (core/process/thread) are only half of it. The other half is the *mechanisms* that connect them — and those are the real prerequisites, because the entire Go scheduler exists to work around their costs. Let me give you the full ladder first, then walk each rung. Read it bottom-up — that's the order reality works in. Here's each rung in depth.

**The core (CPU core).** This is the only thing that physically executes instructions. A core fetches an instruction, runs it, fetches the next — billions of times a second, but fundamentally *one stream of instructions at a time*. A modern CPU has several cores (4, 8, 16...), which is the only reason true simultaneous execution exists at all: 8 cores means at most 8 instruction streams



literally running at the same instant. One wrinkle you'll hear about — *hyperthreading* (SMT) — makes one physical core present itself as two "logical" cores to squeeze a bit more out of idle moments, which is why your laptop reports more "CPUs" than it has physical cores. The thing to lock in: **a core runs exactly one thread at any given moment**. This is the hard ceiling that GOMAXPROCS mirrors.

**The process.** A process is a running program plus the private world the OS gives it: its own address space (its memory — code, heap, stacks), its open file descriptors, its environment. The key word is *isolated* — one process can't reach into another's memory. When you run your Go binary at Probus, that's one process. Everything we talked about — Gs, Ms, Ps, queues — all lives inside that single process's address space. The process is the building; everything else happens inside it.

**The thread (OS thread).** This is the single most important prereq, because it's the unit the operating system actually *schedules*. A process starts with one thread and can spawn more, all sharing the process's memory. The thread is what gets placed onto a core to run. And it's heavy: it carries a large stack (often ~1 MB reserved), plus kernel bookkeeping. In Go, **an M is an OS thread**. When you read "Go multiplexes goroutines onto threads," it means: many cheap Gs take turns running on a few expensive OS threads.

Those are your three nouns. Now the three mechanisms that make the whole design necessary:

**The OS scheduler and context switching.** You have way more threads than cores — your machine right now is running hundreds of threads on 8 cores. The OS scheduler creates the *illusion* that they all run at once by **time-slicing**: it runs thread A for a few milliseconds, then *context-switches* — saves A's registers and program counter, loads B's, and runs B. To you it looks simultaneous; really it's rapid turn-taking. The catch is that a thread context switch is *expensive*: it goes through the kernel, may flush CPU caches, and costs on the order of a microsecond plus lost cache warmth. Do it millions of times a second and you've burned real CPU just on switching. *This cost is the entire reason goroutines exist* — Go switches between goroutines in user space without bothering the kernel, so it's an order of magnitude cheaper.

**Kernel mode vs user mode (the syscall boundary).** See that dashed line in the diagram — it's the concept most people skip and it's the linchpin. The CPU runs in two privilege levels. *User mode* is where your application code runs; it can't directly touch hardware, open files, or talk to the network. *Kernel mode* is where the OS runs with full privileges. When your program needs the OS to do something privileged — open a file, read a socket, create a thread — it makes a **syscall**, which traps from user mode down into kernel mode and back. That round trip is *expensive* (hundreds of nanoseconds to microseconds). This is why it matters that **Go's scheduler runs entirely in user mode**: scheduling a goroutine, parking it, stealing work — none of that crosses into the kernel. Only genuinely privileged operations (a real blocking syscall, creating an M) pay the crossing cost. That dashed line is the toll booth, and Go's whole architecture is about not paying the toll unless it absolutely has to.

**Concurrency vs parallelism (the framing).** These two get used interchangeably and they're not the same. *Concurrency* is structuring a program as independent tasks that can be *in progress* at overlapping times — a single chef juggling five dishes, switching between them, is concurrent with one core. *Parallelism* is tasks literally executing at the same instant — five chefs cooking five dishes needs five cores. Goroutines give you concurrency *always* (you can have a million on one core, taking turns); they give you parallelism *only up to* GOMAXPROCS cores. Rob Pike's line is worth holding onto: concurrency is about *dealing with* many things at once; parallelism is about *doing* many things at once. The Go scheduler is the machine that turns your concurrent goroutines into actual parallelism across the cores you have.

Two smaller supporting pieces, just enough to not be lost later:

**Stack vs heap, registers, and the program counter.** Each running thread (and each goroutine) has a *stack* — a scratch region holding its function calls and local variables, growing and shrinking as functions are called and return. The *heap* is shared, longer-lived memory. *Registers* are the tiny ultra-fast slots inside the core holding the values being worked on right now, and the *program counter* is the register pointing at the current instruction. "Saving a goroutine's state" on a context switch means stashing exactly these — its PC, stack pointer, and registers — so it can resume mid-sentence. You need this to understand why goroutine switching is cheap (a few registers, user space) and you'll need it again for stack-growth and escape analysis.

**Blocking vs non-blocking I/O.** When code asks for data that isn't ready — reading a socket with nothing in it — the naive behavior is to *block*: the thread sits there, occupying a core's scheduling slot, doing nothing until data arrives. With thousands of connections that's thousands of stuck threads. *Non-blocking I/O* instead lets you ask the OS "tell me when this is ready" and go do other work. This is the exact distinction the network poller is built on — and you can't really understand `epoll`/the netpoll story without first feeling why blocking a whole thread per connection is wasteful.

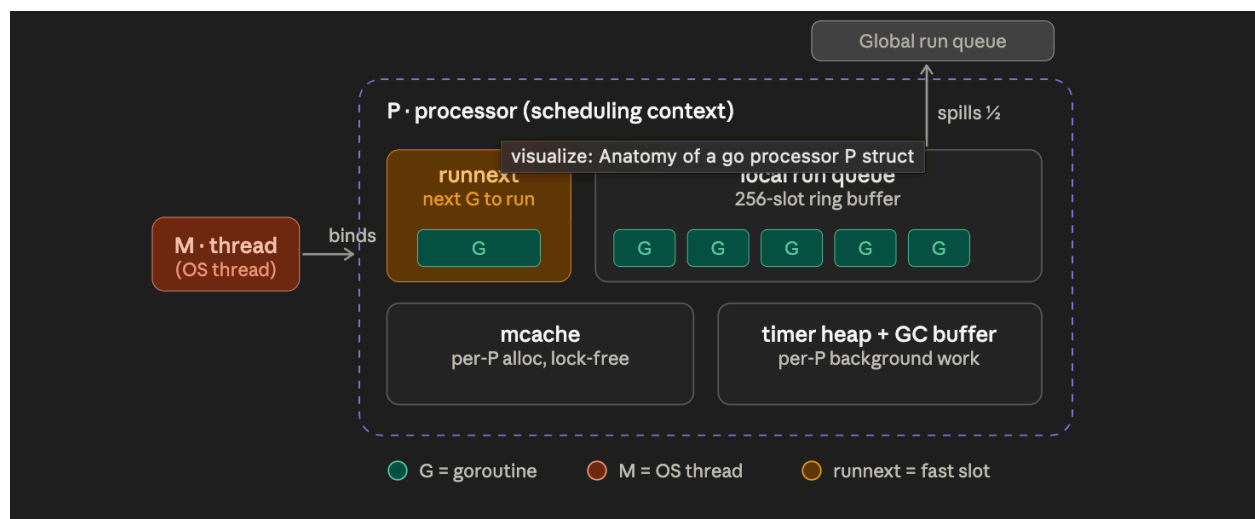
---

So the honest minimum, in dependency order: **core → process → thread → context switching + its cost → user mode vs kernel mode + syscalls → concurrency vs parallelism**, with *stack/heap/registers* and *blocking vs non-blocking I/O* as the two supporting ideas. You named the bottom three correctly; the ones that actually unlock the Go material are the three mechanisms in the middle — especially the **user/kernel boundary**, because every "Go is cheap" claim traces back to Go doing its scheduling on the user side of that line.

If you want to make this concrete rather than abstract, on your work machine run `cat /proc/cpuinfo | grep -c processor` to see your logical core count, then `ps -eLf | wc -l` to see how many threads the OS is juggling across them right now — that gap between the two numbers *is* the OS scheduler's job, and Go's runtime is doing the same trick one level

up. Want me to go deeper on any single rung — the user/kernel boundary is usually the one worth the most time?

Good — P is the one people find slipperiest, because unlike G (the work) and M (a real thread), P is pure abstraction: it's a *permission slip plus a toolbox*. Let me pin down the one practical detail that's changed recently before I lay out the rest. Here's what's actually inside a single P — this is the part most explanations skip, and it's where P stops being mysterious. ## Why P exists at all (the history is the "aha")



In the earliest Go (before 1.1) there was no P — just Gs and Ms, and a *single global run queue* protected by one global lock. Every scheduling decision — pick the next goroutine, put one back, park one — had to grab that one lock. On a multi-core machine, your eight chefs spent their time fighting over a single clipboard instead of cooking. The lock was a wall that capped how well Go scaled across cores.

Go 1.1 (2013) introduced P to break that wall. The insight: give each P its *own* local run queue so the common operation — "grab the next goroutine to run" — touches only that P's queue and needs **no global lock at all**. P also became the thing that *represents* parallelism, deliberately decoupled from the number of threads. That's the reason P feels abstract: it isn't a piece of hardware and it isn't work — it's the resource that grants permission to run Go code, bundled with the private toolkit that makes running it fast. The station, not the chef and not the dish.

## What a P actually owns

A P is a struct, and the diagram above is roughly its important fields:

The **local run queue** is that 256-slot ring buffer. Its owning M pushes and pops from it without locks in the normal case; only work-stealing from *another* P uses atomic operations. This is the lock-free fast path the global-lock design never had.

The **runnext slot** is a beautiful little optimization worth understanding. It's a single-goroutine slot that holds the *most recently made-runnable* G, and the scheduler runs it *before* anything in the main queue. Why? Locality. When goroutine A sends on a channel and wakes goroutine B, B is almost certainly about to use data A just touched — data that's still warm in the CPU cache. Putting B in **runnext** so it runs immediately, instead of at the back of a 200-deep queue, is what makes channel ping-pong and request/response patterns fast. A stealer can take it, but the owner gets priority. It's the dish you keep right on the cutting board because you're about to start it.

The **mcache** is the under-appreciated one. Memory allocation in Go is *per-P*: small-object allocations are served from this cache that belongs to the P, so two goroutines running on different Ps allocate simultaneously without contending on a lock. When an M binds to a P, it allocates through that P's mcache. This is why Go's allocator scales — it's the station's own stock of prepped ingredients, so you're not queuing at the central pantry for every onion.

It also holds **per-P timers** (since Go 1.14, **time.Sleep/After/Ticker** live in a per-P timer heap rather than one global one — same anti-contention motive) and **per-P GC work buffers** so garbage-collection marking batches its work locally too.

The pattern across all of these is identical: *take something that used to be global and contended, and give every P its own copy*. That single idea — P as the unit of "my own private, lock-free stuff" — is most of what P is for.

## GOMAXPROCS is literally the number of Ps

This is the practical payoff. **GOMAXPROCS** doesn't set the number of threads — it sets the number of **Ps**, which is the real ceiling on how many goroutines can be executing Go code at the same instant. You can read it with **runtime.GOMAXPROCS(0)**.

This matters directly for your work at Probus, because the default *changed recently* and it affects anything you run in containers. Historically the default was the host's logical CPU count. As of Go 1.25, the runtime is container-aware on Linux: if a Go process runs inside a container with a CPU limit, GOMAXPROCS defaults to that CPU limit when it's lower than the core count, and it re-checks roughly every 30 seconds and adjusts if the limit changes. A fractional limit is rounded down — a 4.5-core quota yields GOMAXPROCS of 4. Before this, a Go service on a 2-CPU pod scheduled onto a 16-core node would assume it had 16 CPUs, over-schedule, and get throttled by the kernel — hurting tail latency. Two caveats worth knowing: setting the GOMAXPROCS env var (or calling runtime.GOMAXPROCS) disables the cgroup-aware default, and you can also turn it off with GODEBUG=containermaxprocs=0. So if an old deployment

manifest hardcodes `GOMAXPROCS=8` against a 2-CPU limit, the new smarts silently won't apply — worth a `grep` through your manifests.

## M needs P, and P can be handed off

The binding rule: **an M can only run Go code while it holds a P**. That's why the number of Ms can exceed the number of Ps — the extras are Ms parked in blocking syscalls, each having *released* its P so the kitchen keeps cooking. A P moves through a few states: *idle* (sitting on the runtime's idle list waiting for a chef), *running* (bound to an M, actively scheduling), *syscall* (its M dove into a blocking syscall, so the P is detached and grabbable by another M — the handoff we discussed), plus internal *gctest* and *dead* (when you lower GOMAXPROCS, surplus Ps are killed). When a syscalling M returns, it tries to reacquire any free P; if none is available, it dumps its goroutine onto the global queue and goes to sleep.

So the clean mental model to keep: **Ms are how many chefs you've hired** (can balloon during blocking I/O), **Gs are how many dishes are ordered** (millions, mostly parked), but **Ps are how many stoves the kitchen has** — and that number, `GOMAXPROCS`, is what actually bounds your CPU-bound throughput. When you tune a Go service, you're really asking "how many stoves, and do they match the CPU the container actually gave me?"

If you want to see a P being created, bound, and handed off live, run one of your services with `GODEBUG=schedtrace=1000` — the line it prints every second shows the P count and each P's run-queue depth, and you'll watch the syscall handoff happen in real time when your SES or DB calls block. Want me to break down a real `schedtrace` line field by field next?